

HDOC-DAY-17

Menu-Driven C Program Using Switch-Case and User-Defined Functions

Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option with void return type and a simple function call (without call by value/reference). Each function reads the required input, processes it, and displays the result.

- Generate the largest possible number using its digits.
- Generate the smallest possible number using its digits.
- Swap its first and last digits.
- Check whether it is a palindrome number.

Algorithm

1. Start.
2. Display the menu with four digit-based operations.
3. Read the user choice.
4. Use switch(choice) to select the required operation.
5. For choice 1, call largestNumber(). The function reads a positive integer, counts each digit, and displays digits from 9 to 0.
6. For choice 2, call smallestNumber(). The function reads a positive integer, counts each digit, prints the smallest non-zero digit first, then all zeros, followed by the remaining digits in ascending order.
7. For choice 3, call swapFirstLast(). The function reads a positive integer, finds the first and last digits, and forms the number after interchanging them.
8. For choice 4, call checkPalindrome(). The function reads a positive integer, reverses it, and compares the reversed number with the original number.
9. For any other choice, display "Invalid choice".
10. Stop.

C Program

```
#include <stdio.h>

void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void checkPalindrome(void);

int main(void)
{
    int choice;

    printf("MENU\n");
    printf("1. Largest possible number\n");
    printf("2. Smallest possible number\n");
    printf("3. Swap first and last digits\n");
```

```

printf("4. Check palindrome\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1: largestNumber(); break;
    case 2: smallestNumber(); break;
    case 3: swapFirstLast(); break;
    case 4: checkPalindrome(); break;
    default: printf("Invalid choice\n");
}

return 0;
}

void largestNumber(void)
{
    int n, digit, count[10] = {0}, i;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if (n <= 0) { printf("Enter a positive integer\n"); return; }

    while (n > 0) {
        digit = n % 10;
        count[digit]++;
        n = n / 10;
    }
    printf("Largest number = ");
    for (i = 9; i >= 0; i--)
        while (count[i]-- > 0) printf("%d", i);
    printf("\n");
}

void smallestNumber(void)
{
    int n, digit, count[10] = {0}, i;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if (n <= 0) { printf("Enter a positive integer\n"); return; }

    while (n > 0) {
        digit = n % 10;
        count[digit]++;
        n = n / 10;
    }
    printf("Smallest number = ");
    for (i = 1; i <= 9; i++) {
        if (count[i] > 0) {
            printf("%d", i);
            count[i]--;
            break;
        }
    }
    while (count[0]-- > 0) printf("0");
    for (i = 1; i <= 9; i++)
        while (count[i]-- > 0) printf("%d", i);
    printf("\n");
}

void swapFirstLast(void)
{
    int n, temp, first, last, power = 1, result;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if (n <= 0) { printf("Enter a positive integer\n"); return; }
    if (n < 10) { printf("After swapping = %d\n", n); return; }

```

```

last = n % 10;
temp = n;
while (temp >= 10) {
    temp = temp / 10;
    power = power * 10;
}
first = temp;
result = last * power + (n % power - last) + first;
printf("After swapping = %d\n", result);
}

void checkPalindrome(void)
{
    int n, temp, digit, reverse = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if (n <= 0) { printf("Enter a positive integer\n"); return; }

    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        reverse = reverse * 10 + digit;
        temp = temp / 10;
    }
    if (reverse == n)
        printf("%d is a palindrome number\n", n);
    else
        printf("%d is not a palindrome number\n", n);
}

```

COMPIlation

```

(kali㉿kali)-[~/hdoc17]
$ gcc day17.c -o day17

```

```

(kali㉿kali)-[~/hdoc17]
$ ./day17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 1
Enter a positive integer: 35142
Largest possible number: 54321

```

```

(kali㉿kali)-[~/hdoc17]
$ ./day17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 2
Enter a positive integer: 50321
Smallest possible number: 10235

```

```

(kali㉿kali)-[~/hdoc17]
$ ./day17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 3
Enter a positive integer: 12345
Number after swapping: 52341

```

```

(kali㉿kali)-[~/hdoc17]
$ ./day17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome number
Enter your choice: 4
Enter a positive integer: 12321
12321 is a palindrome number.

(kali㉿kali)-[~/hdoc17]
$ █

```

Test Case Table

Test Case	Choice	Input Number	Expected Output	Result
1	1	35142	Largest number = 54321	Pass
2	2	50321	Smallest number = 10235	Pass
3	3	12345	After swapping = 52341	Pass
4	4	12321	12321 is a palindrome number	Pass
5	4	12345	12345 is not a palindrome number	Pass

Result

The menu-driven C program using switch-case and separate void user-defined functions was written, compiled, executed, and tested successfully for the given digit-based operations.